Aperture Photometry with `photutils`

An Exhaustive Guide from First Principles to Full Pipeline

Conceptual foundations drawn from
An Introduction to Astronomical Photometry Using CCDs
W. Romanishin, University of Oklahoma (2002)

Software documentation based on `photutils` v2.3 (Astropy)

April 5, 2026

This document assumes *no prior knowledge* of astronomical photometry or the `photutils` Python library. It begins with the physics of light and telescopes, works through CCDs, calibration, and the complete theory of aperture photometry, before presenting the full `photutils` API with worked code examples.

https://photutils.readthedocs.io/en/stable/

Contents

1 What Is Photometry? 4
- 1.1 The Fundamental Goal . . . 4
- 1.2 Photometry vs. Spectrophotometry . . . 4
- 1.3 The Magnitude System . . . 4
- 1.4 Color Indices and Filter Systems . . . 5

2 Telescopes and Image Formation 5
- 2.1 Why Telescopes? . . . 5
- 2.2 The Point Spread Function . . . 6
- 2.3 Seeing . . . 6

3 The Atmosphere: Its Effects on Photometry 6
- 3.1 Four Major Atmospheric Problems . . . 6
- 3.2 Optical Depth and the Extinction Coefficient . . . 7
- 3.3 Airmass . . . 7

4 CCDs as Photometric Detectors 8
- 4.1 From Photons to Counts . . . 8
- 4.2 Bias, Dark, and Flat Frames . . . 8
  - 4.2.1 Bias Frame . . . 8
  - 4.2.2 Dark Frame . . . 8
  - 4.2.3 Flat Field Frame . . . 8
  - 4.2.4 The Reduction Equation . . . 9
- 4.3 The FITS Image Format . . . 9

5 Aperture Photometry: Theory and Practice 9
- 5.1 The Core Principle . . . 9
- 5.2 The PSF Has No Edge . . . 9
- 5.3 The Aperture Size Trade-off . . . 10
- 5.4 The Background Annulus . . . 10
- 5.5 The Aperture Correction Technique . . . 10
- 5.6 Signal-to-Noise Ratio: The Full Derivation . . . 11
  - 5.6.1 Full noise budget including read noise and dark current . . . 11
  - 5.6.2 Scaling with exposure time and telescope aperture . . . 12
  - 5.6.3 Limiting magnitude . . . 12
- 5.7 All-Sky vs. Differential Photometry . . . 12
- 5.8 Standard Stars and Calibration . . . 12

6 Introduction to `photutils` 13
- 6.1 What Is `photutils`? . . . 13
- 6.2 Installation . . . 13
- 6.3 The Astropy Ecosystem . . . 13

7 Step 1: Loading and Inspecting Your Image 13

8 Step 2: Background Estimation and Subtraction 14
- 8.1 Scalar Background (Flat, Uniform Sky) . . . 14

8.2 2D Background (Spatially Varying Sky) . . . 14
8.2.1 How Background2D Works . . . 15
8.2.2 Available Background Estimators . . . 15

9 Step 3: Defining Apertures **15**
9.1 Pixel-Coordinate Apertures . . . 16
9.2 Sky-Coordinate Apertures . . . 16
9.3 Complete Aperture Class Reference . . . 17
9.4 Visualising Apertures . . . 17

10 Step 4: Pixel Overlap Methods **17**

11 Step 5: Running Aperture Photometry **18**
11.1 `aperture_photometry()` — Simple Flux Sums . . . 18
11.1.1 Full Signature . . . 18
11.1.2 Output QTable . . . 19
11.1.3 Minimal Working Example . . . 19
11.1.4 Multiple Apertures at the Same Positions . . . 19
11.1.5 Including Error Propagation . . . 20
11.2 `ApertureStats` — Full Statistical Characterisation . . . 20
11.2.1 Class Signature . . . 20
11.2.2 Key Properties . . . 20
11.2.3 Example . . . 21
11.2.4 Centroid Re-centring . . . 22

12 Step 6: Local Background Estimation with Annuli **22**
12.1 The `local_bkg` Keyword . . . 23

13 Step 7: Error Propagation in Detail **23**

14 Step 8: Masks and Bad Pixels **24**

15 Step 9: WCS and Sky Coordinates **24**

16 Step 10: Converting to Calibrated Magnitudes **24**

17 Full End-to-End Pipeline **25**

18 Common Pitfalls **28**

19 Quick Reference **29**
19.1 Tool Selection . . . 30
19.2 Aperture Size Guidelines (Summary from Romanishin) . . . 30

20 Further Reading **30**

1 What Is Photometry?

1.1 The Fundamental Goal

To do real science about celestial objects, one needs far more than pictures. As Romanishin puts it, "observational astronomy becomes science only when we can start to answer questions quantitatively: how far away is that object? How much energy does it emit? How hot is it?"

The most fundamental piece of information we can measure about objects beyond our solar system is the amount of energy in the form of electromagnetic radiation (EMR) that we receive from them. This quantity is called the **flux**, defined as the energy received per unit time per unit area perpendicular to the line of sight. In cgs units, spectral flux is expressed as:

$$f_\lambda = \mathrm{erg\,s^{-1}\,cm^{-2}\,\text{Å}^{-1}} \qquad \text{or} \qquad f_\nu = \mathrm{erg\,s^{-1}\,cm^{-2}\,Hz^{-1}} \tag{1}$$

Photometry is the science of measuring this flux (or quantities directly proportional to it) from celestial objects. When coupled with a distance estimate, a flux measurement gives us the total energy output of the object (its **luminosity**), its temperature, and other physical properties.

1.2 Photometry vs. Spectrophotometry

A **spectrophotometric** observation uses a dispersing element (diffraction grating or prism) to spread light into its component wavelengths, providing the spectral energy distribution (SED) of the source. This delivers the most information per observation, but it dilutes the light over many wavelength bins, and requires a spectrograph — a complex, expensive piece of equipment.

Filter photometry (usually called simply "photometry") restricts the wavelengths measured by placing a filter with a broad passband in front of the detector. No wavelength resolution is lost, so all photons within the passband land on the same pixel, yielding far higher sensitivity for faint sources. The tradeoff is less wavelength information. A filter with a passband of $\sim 1000\,\text{Å}$ has a resolution $R = \lambda/\Delta\lambda$ of only 5–20, so filter photometry is essentially very low resolution spectrophotometry. The great advantage: "it is much easier to make photometric observations of faint objects than it is to make spectroscopic observations of the same object."

1.3 The Magnitude System

Optical astronomers almost universally express brightness in **magnitudes**, a logarithmic scale with ancient roots. Hipparchus ($\sim$190 BCE) divided naked-eye stars into six brightness classes; later measurement showed these classes corresponded to approximately a factor of 2.5 in flux per step. The modern definition is exact: a difference of 5 magnitudes corresponds to a flux ratio of exactly 100. Therefore each magnitude step is $100^{1/5} \approx 2.512$ in flux.

For any two sources with fluxes f_1 and f_2, the magnitude difference is:

$$m_1 - m_2 = -2.5 \log_{10}\left(\frac{f_1}{f_2}\right) \tag{2}$$

This is the *fundamental equation* of the magnitude system. Rearranging gives the flux ratio from a known magnitude difference:

$$\frac{f_1}{f_2} = 10^{-0.4(m_1 - m_2)} \tag{3}$$

To assign an absolute magnitude to any star, a zero-point must be chosen. Historically, the star Vega was defined to have magnitude 0.00 in all filters (in practice the modern standard uses the average of six A0 V stars; Vega itself has $m \approx 0.03$). Thus:

$$m_1 = -2.5 \log_{10}\left(\frac{f_1}{f_{\text{Vega}}}\right) \tag{4}$$

Magnitudes are a shorthand for flux ratios. Brighter objects have *smaller* (or more negative) magnitudes. The Sun is $m_V = -26.7$; the faintest naked-eye stars are $\approx +6$; modern large telescopes reach $+30$ and beyond. Importantly, because magnitudes are logarithms, you *cannot* simply add or subtract magnitudes to combine fluxes — you must convert back to flux first.

1.4 Color Indices and Filter Systems

The **color** of an object describes how its flux varies with wavelength. Using broadband filters, astronomers define **color indices** as the magnitude difference between two passbands:

$$B - V = m_B - m_V = -2.5 \log(f_B/f_V) + \text{const} \tag{5}$$

The constant arises because the zero-point is defined by A-type stars (Vega), for which $B - V \equiv 0.00$. A blue, hot star might have $B - V \approx -0.3$; the Sun has $B - V \approx +0.67$; the very red supergiant Betelgeuse has $B - V \approx +2.0$.

The most widely used optical system is **UBVRI**, with approximate central wavelengths: U ≈ 3600 Å, B ≈ 4400 Å, V ≈ 5500 Å, R ≈ 6400 Å, I ≈ 8000 Å.

2 Telescopes and Image Formation

2.1 Why Telescopes?

Telescopes serve four functions: (1) they collect far more light than the unaided eye, allowing observation of much fainter objects; (2) they have increased angular resolution; (3) attached detectors allow study of wavelengths invisible to the eye; and (4) detectors provide a permanent, quantitative record.

2.2 The Point Spread Function

A star is for all practical purposes a **point source** — its angular diameter is far smaller than the resolution of any ground-based optical telescope. Yet the image of a star on a CCD is not a point, but a smooth, extended blob. This shape is called the **Point Spread Function** (PSF).

Even above the atmosphere (as with the Hubble Space Telescope), diffraction by the telescope aperture of diameter D spreads a point source into an **Airy disk**, with the radius of the first dark ring given by:

$$\Theta_{\text{Airy}} = \frac{1.22\lambda}{D} \tag{6}$$

This is the **diffraction limit**, also called the Dawes limit.

For a 1-meter telescope at 5500 Å, $\Theta_{\text{Airy}} \approx 0.14$ arcsec. In principle this allows very fine detail. In practice, from the ground, the PSF is *completely dominated* by the atmosphere.

2.3 Seeing

Seeing is the blurring and shimmering of stellar images caused by the passage of light through turbulent pockets of air at different temperatures. Since the index of refraction of air depends on temperature, temperature variations along the line of sight continuously deflect rays, causing the stellar image to wander and blur. The result is a fuzzy blob rather than an Airy pattern.

Seeing is characterized by the **Full Width at Half Maximum** (FWHM) of the stellar profile — the diameter of the image at the level of half its peak intensity. At excellent sites (e.g. Mauna Kea, 4200 m elevation), the seeing can be as good as 0.5 arcsec FWHM. At a typical university campus observatory, 2–3 arcsec FWHM is more representative.

> **Critical consequence for photometry:** Because seeing blurs star images over many pixels, we must sum counts over an *aperture* covering the entire PSF — not just a single pixel — to recover the total flux. And because the PSF has no sharp edge (it fades smoothly to zero), we face a fundamental question: *how large should the aperture be?*

3 The Atmosphere: Its Effects on Photometry

3.1 Four Major Atmospheric Problems

The atmosphere creates four distinct problems for photometrists:

(1) Seeing (discussed above): blurs stellar images, degrading resolution and making flux measurement in small apertures less reliable.

(2) Skyglow: even at pristine, dark sites, the night sky glows due to atomic excitations in the upper atmosphere (airglow) and, near cities, scattered artificial light. This sky signal adds noise to every measurement. Its brightness — typically expressed in magnitudes

per square arcsecond — is the dominant noise source for faint object photometry. The darkest mountaintop sites reach $V \approx 21.9\,\mathrm{mag\,arcsec^{-2}}$; typical mid-latitude sites are several magnitudes brighter.

(3) Atmospheric Extinction: the atmosphere absorbs and scatters a fraction of all optical light. The flux f_{obs} reaching the telescope is related to the flux f_{inc} above the atmosphere by:

$$f_{\mathrm{obs}} = f_{\mathrm{inc}}\, e^{-\tau} \tag{7}$$

where τ is the optical depth of the atmosphere along the line of sight.

(4) Atmospheric Refraction: the atmosphere acts as a weak prism, bending light toward the zenith and spreading a star into a tiny spectrum. This can smear images, particularly when observing at large zenith angles through a broad filter.

3.2 Optical Depth and the Extinction Coefficient

For a beam traversing an absorbing medium of optical depth τ:

$$f_{\mathrm{out}} = f_{\mathrm{in}}\, e^{-\tau} \tag{8}$$

Optical depths add linearly: looking through twice as much air doubles τ, even though the transmitted fraction does not halve.

Photometrists characterize atmospheric opacity by the **extinction coefficient** K, expressed in magnitudes per airmass:

$$K = 2.5\,\log_{10}\left(\frac{f_{\mathrm{inc}}}{f_{\mathrm{obs}}(\theta = 0)}\right) \qquad \text{(at zenith)} \tag{9}$$

From the instrumental magnitude $m_I = -2.5\log(\mathrm{counts\,s^{-1}})$, the slope of m_I vs. airmass X gives K directly:

$$K = \frac{\Delta m_I}{\Delta X} \tag{10}$$

Typical values at a 2200 m site: $K_V \approx 0.14$, $K_B \approx 0.26$, $K_R \approx 0.10\,\mathrm{mag\,airmass^{-1}}$.

3.3 Airmass

The **airmass** X is the amount of atmosphere along the line of sight relative to looking straight up. For a plane-parallel atmosphere at zenith angle θ_Z:

$$X \approx \sec\theta_Z = \frac{1}{\sin\lambda\sin\delta + \cos\lambda\cos\delta\cos h} \tag{11}$$

where λ is the observatory latitude, δ the declination, and h the hour angle. A small curvature correction applies near the horizon:

$$X = \sec\theta_Z - \Delta X, \quad \Delta X = 0.00186(\sec\theta_Z - 1) + 0.002875(\sec\theta_Z - 1)^2 + \ldots \tag{12}$$

The to-zero-airmass correction to an instrumental magnitude observed at airmass X is:

$$m_I^{0\mathrm{AM}} = m_I - X\,K_\lambda \tag{13}$$

4 CCDs as Photometric Detectors

4.1 From Photons to Counts

A **Charge-Coupled Device** (CCD) is a two-dimensional array of photosensitive silicon pixels. Each pixel independently collects electrons liberated by incoming photons via the photoelectric effect. After the exposure, the charge in each pixel is read out sequentially as a voltage, amplified, digitized, and stored as an integer number.

The chain from source to data number (DN, also called ADU for Analog-to-Digital Unit) involves several conversion factors:

$$\text{Photons} = \frac{\text{gain} \times \text{DN}}{\text{QE}} \tag{14}$$

Quantum Efficiency (QE) is the fraction of incident photons that actually liberate an electron. Modern scientific CCDs have peak QEs of 70–95% in the red/green, but may drop significantly in the blue (especially for older "thick" or front-illuminated chips). "Thin" or back-illuminated chips have much better blue QE.

Gain (g, in electrons/ADU) converts the electron count to data numbers. Typical values are 1–5 e^-/ADU.

Read Noise (R, in electrons/pixel) is the irreducible electronic noise introduced each time the CCD is read out. Modern CCDs have read noise of 3–15 e^- per pixel per readout.

4.2 Bias, Dark, and Flat Frames

Three types of calibration frames are essential before any photometry can be performed:

4.2.1 Bias Frame

A **bias frame** is a zero-second exposure: the CCD is read out without any integration. It measures the inherent electronic offset that every CCD readout adds to the signal. Multiple bias frames are median-combined to produce a low-noise master bias.

4.2.2 Dark Frame

Even in complete darkness, thermal excitation of electrons in the silicon creates a slowly accumulating **dark current** signal. A dark frame is an exposure of the same duration as the science frame, with the shutter closed. Modern professional CCDs cooled to −100 °C have negligible dark current; amateur thermoelectrically cooled CCDs operated at −20 to −40 °C may have significant dark current.

4.2.3 Flat Field Frame

CCD pixels are not perfectly uniform: they vary in size, in silicon thickness, and in the degree to which the optics illuminate each one (vignetting). These effects can produce pixel-to-pixel sensitivity variations of several percent. A **flat field** is an image of a uniformly illuminated surface (twilight sky or dome screen), used to map and correct these variations. It is divided into the data frame.

4.2.4 The Reduction Equation

The complete calibration pipeline for professional CCDs is:

$$D_{\text{cal}} = \frac{D_{\text{raw}} - B_{\text{bias}} - D_{\text{dark}}}{F_{\text{flat,norm}}} \tag{15}$$

where $F_{\text{flat,norm}}$ is the flat field divided by its mean (so its mean is 1.0). After this step, D_{cal} is linearly proportional to the number of photons that fell on each pixel.

Warning on flat fielding and photometry: Scattered light — light reaching the CCD without passing through the full optical train — must be carefully excluded from flat fields. If the flat includes scattered light, it will *not* properly correct the images of stars, which receive only light through the complete optical system. As Romanishin warns: "Don't just assume that because your images look nice that they are photometrically correct."

4.3 The FITS Image Format

Calibrated images are almost universally stored in **FITS** (Flexible Image Transport System) format. A FITS file contains a header (ASCII metadata: telescope, filter, exposure time, observer, WCS coordinates, etc.) followed by the binary pixel data. In Python, FITS files are handled with `astropy.io.fits`.

5 Aperture Photometry: Theory and Practice

5.1 The Core Principle

Aperture photometry is the most direct method of measuring the flux of an isolated source. It requires no model of the PSF shape. The idea, as described by Romanishin, is: the CCD image is "simply a two-dimensional set of numbers, one number for each pixel. After the initial reduction steps (bias and dark subtraction, division by a flat field) the number at each pixel should be linearly related to the number of photons that fell on that pixel."

The instrumental magnitude is then defined as:

$$m_I = -2.5 \log_{10}\left(\frac{N_{\text{ap}} - A_{\text{ap}}\, S_{\text{sky}}}{t_{\text{exp}}}\right) \tag{16}$$

where N_{ap} is the total counts in the measurement aperture, A_{ap} is the area of the aperture in pixels, S_{sky} is the sky signal per pixel (measured from a nearby annulus), and t_{exp} is the exposure time in seconds.

5.2 The PSF Has No Edge

A critical subtlety: "the PSF does not have an edge. The intensity of the star fades smoothly to zero with increasing radius, but there is no place that we could call an 'edge'." The PSF extends theoretically to infinity. This raises the unavoidable question: *how large must the aperture be to capture all the flux?*

5.3 The Aperture Size Trade-off

Choosing the aperture radius $r_{\rm ap}$ involves competing demands:

Arguments for a large aperture:

- Captures more of the source flux (more of the PSF wings).
- Less sensitive to pointing jitter and tracking errors.
- Less sensitive to seeing variations (more of the PSF is always enclosed).

Arguments for a small aperture:

- Less sky area ⇒ less sky noise in the aperture.
- Smaller chance of including light from nearby contaminating sources.
- Better signal-to-noise for faint objects.

The balance point is approximately $r_{\rm ap} \approx 1.0$–$1.5 \times$ FWHM. At this radius, studies (Howell 1989; Harris 1990) show that "the S/N does not appear to be too sensitive to the exact small aperture size," and the optimal aperture diameter is close to $1.4 \times$ FWHM.

Practical rule: Use an aperture of radius ∼1.0–1.5×FWHM for optimal SNR on faint sources in uncrowded fields, and apply an aperture correction (see below) to account for the flux in the PSF wings. For bright sources in isolation, a larger aperture (∼4×FWHM) minimises aperture correction but increases sky noise.

5.4 The Background Annulus

Every pixel in the aperture contains both source photons and sky photons. Sky photons must be estimated and subtracted. The standard procedure is to measure the sky level in a **circular annulus** centered on the source, with inner radius $r_{\rm in}$ and outer radius $r_{\rm out}$, where $r_{\rm in} > r_{\rm ap}$ by a comfortable margin of at least 2–3 pixels. The sky per pixel $S_{\rm sky}$ is estimated from the statistics of the annulus pixels (see Equation 16).

Key guidelines for the annulus:

- Typical: $r_{\rm in} \approx 4$–$5 \times r_{\rm ap}$, $r_{\rm out} \approx 6$–$7 \times r_{\rm ap}$.
- The annulus must be large enough to contain many pixels for good statistics.
- **Use the median, not the mean**, of annulus pixels. The mean is biased upward by faint undetected sources and cosmic rays; "the pixel values can be sufficiently skewed to result in the mean giving an estimate of the sky too poor for high precision photometry."
- Apply **sigma clipping** to reject outlier pixels in the annulus before computing the sky level.

5.5 The Aperture Correction Technique

For faint sources, using a large aperture to capture all the PSF light would include so much sky that the S/N becomes unacceptably low. The **aperture correction** technique

solves this elegantly.

All stars in the same image have the same PSF (the same atmospheric blurring affects every star). Therefore, the ratio of flux measured in a small aperture to the total flux is identical for every star in the frame (as long as the seeing does not change within the exposure, which is generally valid for exposures shorter than ∼30 min).

The procedure is:

1. Choose a *small* aperture (radius ≈ 1×FWHM) for measuring the faint target. This maximises S/N.
2. Find a bright, isolated star in the same frame. Measure its magnitude in both the small aperture $m_I(r_{\text{small}})$ and a large aperture $m_I(r_{\text{large}})$ that captures essentially all the light.
3. The aperture correction is:
$$\Delta = m_I(r_{\text{large}}) - m_I(r_{\text{small}}) \tag{17}$$
Δ is always negative (more light in the bigger aperture).
4. Apply to the faint target:
$$m_I^{\text{total}} = m_I^{\text{small}}(\text{faint}) + \Delta \tag{18}$$

At the optimal small aperture diameter of $\sim 1.4\times$FWHM, the aperture correction is approximately $\Delta \approx -0.3$ mag.

5.6 Signal-to-Noise Ratio: The Full Derivation

The S/N of a photometric measurement is the most important quantity characterising its quality. The noise has multiple contributions.

Let C_{star} be the counts from the source alone, and C_{sky} be the total sky counts in the measurement aperture. Both can only be measured together; the sky is estimated from a separate sky aperture. Using standard error propagation (Romanishin, Chapter 20):

$$N = \sqrt{C_{\text{star}} + 2\,C_{\text{sky}}} \tag{19}$$

so the S/N is:

$$\frac{S}{N} = \frac{C_{\text{star}}}{\sqrt{C_{\text{star}} + 2\,C_{\text{sky}}}} \tag{20}$$

The factor of 2 arises because we observe the sky *twice*: once along with the star, and once in the separate sky aperture. On a CCD with a large sky annulus, this factor approaches 1.

5.6.1 Full noise budget including read noise and dark current

In the general case, the noise over an aperture of N_{pix} pixels is:

$$\sigma_{\text{total}}^2 = C_{\text{star}} + N_{\text{pix}}\left(C_{\text{sky}} + D \cdot t + R^2\right) \tag{21}$$

where D is the dark current rate (electrons/pixel/second), t is the exposure time, and R is the read noise (electrons/pixel). For well-cooled modern CCDs and broadband filters on faint sources, sky noise typically dominates.

5.6.2 Scaling with exposure time and telescope aperture

Writing $C = t \cdot R_{\text{count}}$ for each contributing term:

$$\frac{S}{N} \propto \sqrt{t} \tag{22}$$

Doubling the S/N requires *four times* the exposure time. Similarly, doubling the telescope diameter D_{tel} quadruples the collecting area and increases the S/N by a factor of 2:

$$\frac{S}{N} \propto D_{\text{tel}} \tag{23}$$

5.6.3 Limiting magnitude

For faint objects where $C_{\text{star}} \ll C_{\text{sky}}$, the noise is dominated entirely by the sky. In this background-limited regime:

$$\frac{S}{N} \approx \frac{C_{\text{star}}}{\sqrt{2\,C_{\text{sky}}}} \tag{24}$$

"The fact that we are counting photons with a CCD, and that we understand the noise characteristics of photons" makes it possible to calculate the limiting magnitude of any given telescope-CCD combination from first principles.

5.7 All-Sky vs. Differential Photometry

All-sky photometry compares an object to standard stars in a completely different part of the sky. It requires perfectly photometric (cloudless) conditions throughout the night. Extinction must be carefully measured and applied.

Differential photometry compares an object to comparison stars *on the same CCD frame*, observed simultaneously. Thin clouds or minor atmospheric variations affect all objects in the frame equally, so the ratio of fluxes is preserved. This mode is ideal for variable star monitoring and can be performed even from sites with variable sky transparency. "This ability to do differential photometry under non-photometric skies is one of the real advantages of the CCD."

5.8 Standard Stars and Calibration

Absolute calibration requires **standard stars** — objects with precisely known magnitudes in the chosen photometric system. In practice, one measures the *instrumental* magnitudes of the target and standard stars, then relates them via **transformation equations**. For a two-filter system (V and R):

$$(V - R) = a\,(V - R)_I^{0\text{AM}} + b \tag{25}$$

$$V = m_I^{0\text{AM}}(V) + V_0^{\text{ZP}} + c\,(V - R) \tag{26}$$

where a, b, c, and V_0^{ZP} are determined from observations of Landolt standard stars spanning a wide range of colors. Ideally $a \approx 1.00$ and $b \approx 0.00$, indicating that the instrumental system closely matches the standard system.

6 Introduction to photutils

6.1 What Is photutils?

photutils is an open-source Python library and **coordinated package of Astropy**, providing tools for source detection and photometric measurement of astronomical sources in digital images. It integrates seamlessly with the Astropy ecosystem and follows Astropy conventions for data containers, units, and WCS.

photutils provides: background estimation, source finding and detection, aperture photometry (this guide), PSF photometry, image segmentation, centroids, radial profiles, and isophote fitting.

6.2 Installation

```
pip install photutils           # via pip
conda install -c conda-forge photutils   # via conda
```

Key dependencies: `numpy`, `astropy`, `scipy`. Optional: `bottleneck` (faster NaN-aware statistics).

6.3 The Astropy Ecosystem

In a typical photometry pipeline you will use:

- `astropy.io.fits` — read/write FITS images.
- `astropy.wcs.WCS` — pixel ↔ sky coordinate transforms.
- `astropy.stats.SigmaClip`, `sigma_clipped_stats` — robust statistics.
- `astropy.coordinates.SkyCoord` — sky coordinate objects.
- `astropy.units` — physical units.
- `astropy.nddata.NDData` — data containers with embedded metadata.
- `astropy.table.QTable` — tables with units (returned by photometry functions).

7 Step 1: Loading and Inspecting Your Image

```
from astropy.io import fits
import numpy as np

# Open a FITS file
with fits.open('science_image.fits') as hdul:
    hdul.info()                        # Inspect structure
    data   = hdul[0].data.astype(float).copy()
    header = hdul[0].header.copy()

```

```
print(f"Shape: {data.shape}  Min: {data.min():.1f}  Max: {data
    .max():.1f}")

# Read WCS (World Coordinate System) for sky coordinates
from astropy.wcs import WCS
wcs = WCS(header)
```

Always `.copy()` the data. By default, `hdul[n].data` returns a memory-mapped array linked to the file on disk. If you modify it (e.g., subtract the background) without copying first, you risk modifying the original file or encountering errors after closing the file handle.

8 Step 2: Background Estimation and Subtraction

Both `aperture_photometry()` and `ApertureStats` require **background-subtracted data**. This is a hard requirement; skipping it leads to systematically inflated flux values.

8.1 Scalar Background (Flat, Uniform Sky)

For images where the background is approximately constant across the field, a single scalar value suffices:

```
from astropy.stats import sigma_clipped_stats

# Sigma-clip to reject sources and cosmic rays, then measure
    sky
mean, median, std = sigma_clipped_stats(data, sigma=3.0)

print(f"Background median: {median:.4f} ADU/pixel")
print(f"Background std:    {std:.4f} ADU/pixel")

# Subtract scalar background
data_bkgsub = data - median
```

The **median** is strongly preferred over the mean. Sources and cosmic rays add a positive tail to the pixel intensity distribution; the mean of this skewed distribution is biased high, while the median is robust.

8.2 2D Background (Spatially Varying Sky)

For real images with sky gradients (scattered light, astrophysical backgrounds, variable illumination), a 2D background model is necessary:

```
from astropy.stats import SigmaClip
from photutils.background import Background2D,
    MedianBackground

sigma_clip     = SigmaClip(sigma=3.0)
```

```python
bkg_estimator = MedianBackground()

# Background2D divides image into boxes, computes sky in each box,
# median-filters the map, then interpolates to original resolution.
bkg = Background2D(data,
                   box_size=(64, 64),     # boxes must be larger than typical sources
                   filter_size=(3, 3),    # smooth the low-res map
                   sigma_clip=sigma_clip,
                   bkg_estimator=bkg_estimator)

print(f"Median background:     {bkg.background_median:.4f}")
print(f"Median background RMS: {bkg.background_rms_median:.4f}")

data_bkgsub = data - bkg.background      # subtract 2D background image
```

8.2.1 How Background2D Works

The algorithm follows the same principle as the sky annulus technique applied globally: the image is divided into boxes; sigma-clipped statistics are computed in each box (rejecting sources and hot pixels); the low-resolution map is median-filtered; and finally interpolated (default: bicubic spline) back to the original image size.

8.2.2 Available Background Estimators

Class	Method
`MeanBackground`	Arithmetic mean of sigma-clipped pixels.
`MedianBackground`	Median. Robust; recommended default.
`ModeEstimatorBackground`	$3 \times$ median $- 2 \times$ mean.
`MMMBackground`	Similar to above with additional fallback.
`SExtractorBackground`	$2.5 \times$ median $- 1.5 \times$ mean; falls back to median if distribution is too skewed. *photutils* default.
`BiweightLocationBackground`	Biweight location (very robust M-estimator).

9 Step 3: Defining Apertures

9.1 Pixel-Coordinate Apertures

All pixel-based aperture classes take `positions` as their first argument: a single `(x, y)` tuple, a list of tuples, or an $N \times 2$ NumPy array. Note that `photutils` uses 0-indexed (x, y) pixel coordinates (Python convention).

```
import numpy as np
from photutils.aperture import (CircularAperture, CircularAnnulus,
                                EllipticalAperture, EllipticalAnnulus,
                                RectangularAperture, RectangularAnnulus)

positions = [(145.1, 168.3), (84.5, 224.1), (48.3, 200.3)]

# Circular aperture: radius r (pixels)
src_ap  = CircularAperture(positions, r=5.0)

# Circular annulus: inner radius r_in, outer radius r_out (pixels)
bkg_ann = CircularAnnulus(positions, r_in=10.0, r_out=15.0)

# Elliptical aperture: semi-major a, semi-minor b, rotation theta (radians)
ell_ap  = EllipticalAperture(positions, a=8.0, b=4.0, theta=np.pi/4)

# Rectangular aperture: full width w, height h, rotation theta
rect_ap = RectangularAperture(positions, w=10.0, h=6.0, theta=0.0)
```

9.2 Sky-Coordinate Apertures

Each pixel aperture has a sky counterpart (prefixed `Sky`). These accept `SkyCoord` positions and angular sizes:

```
from astropy.coordinates import SkyCoord
from astropy import units as u
from photutils.aperture import SkyCircularAperture, SkyCircularAnnulus

pos_sky = SkyCoord(ra=[83.82]*u.deg, dec=[-5.39]*u.deg)
sky_ap  = SkyCircularAperture(pos_sky, r=3.0*u.arcsec)
sky_ann = SkyCircularAnnulus(pos_sky, r_in=5.0*u.arcsec, r_out=8.0*u.arcsec)

# Sky apertures require wcs= when performing photometry
```

9.3 Complete Aperture Class Reference

Pixel Class	Sky Class
CircularAperture	SkyCircularAperture
CircularAnnulus	SkyCircularAnnulus
EllipticalAperture	SkyEllipticalAperture
EllipticalAnnulus	SkyEllipticalAnnulus
RectangularAperture	SkyRectangularAperture
RectangularAnnulus	SkyRectangularAnnulus

9.4 Visualising Apertures

Always overlay your apertures on the image before running photometry:

```
import matplotlib.pyplot as plt
from astropy.visualization import ZScaleInterval

z = ZScaleInterval()
vmin, vmax = z.get_limits(data_bkgsub)

fig, ax = plt.subplots(figsize=(10, 8))
ax.imshow(data_bkgsub, origin='lower', vmin=vmin, vmax=vmax,
    cmap='gray')
src_ap.plot(ax=ax, color='cyan', lw=1.5, label='Source
    aperture')
bkg_ann.plot(ax=ax, color='red',  lw=1.5, label='Background
    annulus')
ax.legend()
plt.show()
```

10 Step 4: Pixel Overlap Methods

When a circular (or any curved) aperture is overlaid on a pixel grid, many pixels lie partially inside and partially outside the aperture. The `method` keyword controls how these boundary pixels are handled.

method	Pixel weights	Description
exact	$[0, 1]$ fractional	Exact analytical fractional overlap. Most accurate; slowest. Default.
center	0 or 1	Pixel fully included if centre is inside aperture; excluded otherwise. Fastest.
subpixel	$[0, 1]$	Each pixel subdivided into subpixels^2 sub-pixels; each sub-pixel is in or out based on its centre. Higher subpixels → better approximation.

The aperture sum is:

$$F = \sum_i w_i \cdot D_i, \qquad w_i \in [0, 1] \tag{27}$$

Rectangular apertures with method='exact': An internal subpixel approximation (subdividing by 1024) is used. For apertures 2–100 pixels wide, the error is typically below 0.001%. For apertures smaller than ∼1 pixel, use method='subpixel' with a larger subpixels value.

11 Step 5: Running Aperture Photometry

11.1 aperture_photometry() — Simple Flux Sums

11.1.1 Full Signature

```
from photutils.aperture import aperture_photometry

phot_table = aperture_photometry(
    data,              # 2D array: must be background-subtracted
    apertures,         # Aperture object or list of Aperture objects
    error=None,        # 2D per-pixel 1-sigma uncertainty array
    mask=None,         # Boolean mask (True = bad pixel)
    method='exact',    # Pixel overlap method
    subpixels=5,       # Used only if method='subpixel'
    wcs=None,          # Required for sky-coordinate apertures
)
```

11.1.2 Output QTable

Column	Type	Description
id	int	Sequential source identifier (1, 2, ...)
xcenter, ycenter	float	x and y pixel coordinates of aperture centre
sky_center	SkyCoord	Sky coordinates (returned only if wcs provided)
aperture_sum	float	Weighted sum of pixel values inside the aperture
aperture_sum_err	float	Propagated uncertainty (returned only if error provided)

11.1.3 Minimal Working Example

```
import numpy as np
from astropy.stats import sigma_clipped_stats
from photutils.aperture import CircularAperture,
    aperture_photometry
from photutils.datasets import make_100gaussians_image

data = make_100gaussians_image()   # synthetic test image,
    background ~ 5

_, bkg_median, _ = sigma_clipped_stats(data, sigma=3.0)
data_bkgsub = data - bkg_median

positions = [(145.1, 168.3), (84.5, 224.1), (48.3, 200.3)]
aperture  = CircularAperture(positions, r=5.0)

phot_table = aperture_photometry(data_bkgsub, aperture)
phot_table['aperture_sum'].info.format = '%.4f'
print(phot_table)
```

11.1.4 Multiple Apertures at the Same Positions

Passing a list of apertures measures flux in all of them simultaneously, returning columns named aperture_sum_0, aperture_sum_1, etc.:

```
from photutils.aperture import CircularAnnulus

annulus  = CircularAnnulus(positions, r_in=10.0, r_out=15.0)
phot_both = aperture_photometry(data_bkgsub, [aperture,
    annulus])
print(phot_both.colnames)
# ['id', 'xcenter', 'ycenter', 'aperture_sum_0', '
    aperture_sum_1']
```

11.1.5 Including Error Propagation

The error array should represent the total per-pixel 1-σ uncertainty. The `calc_total_error()` utility combines background RMS with Poisson source noise:

$$\sigma_{\text{tot}} = \sqrt{\sigma_{\text{bkg}}^2 + D_{\text{source}}/g} \tag{28}$$

```
from photutils.utils import calc_total_error
from photutils.background import Background2D,
    MedianBackground
from astropy.stats import SigmaClip

bkg = Background2D(data, (64, 64), filter_size=(3, 3),
                   sigma_clip=SigmaClip(sigma=3.0),
                   bkg_estimator=MedianBackground())

data_bkgsub = data - bkg.background
gain        = 2.5              # electrons/ADU for your
    detector
error = calc_total_error(data_bkgsub, bkg.background_rms,
                         effective_gain=gain)

phot = aperture_photometry(data_bkgsub, aperture, error=error)
print(phot['aperture_sum', 'aperture_sum_err'])
```

11.2 ApertureStats — Full Statistical Characterisation

11.2.1 Class Signature

```
from photutils.aperture import ApertureStats

aperstats = ApertureStats(
    data,                  # 2D background-subtracted array
    aperture,              # Aperture object (may hold many
    positions)
    error=None,            # 2D error array
    mask=None,             # Boolean mask
    wcs=None,              # WCS for sky-coordinate apertures
    sigma_clip=None,       # SigmaClip instance (for robust
    stats)
    sum_method='exact',    # Overlap method for sum properties
    subpixels=5,
    local_bkg=None,        # 1D array of per-source background
    values
)
```

Properties are computed using **lazy evaluation**: a property is only calculated the first time it is accessed.

11.2.2 Key Properties

Property	Description
xcentroid, ycentroid	Intensity-weighted centroid in x and y.
centroid	[xcentroid, ycentroid] as an array.
sky_centroid	Sky-coordinate centroid (requires WCS).
sum	Weighted sum of pixel values (the aperture flux).
sum_err	Propagated uncertainty in sum.
sum_aper_area	Effective area (pixels) used in the sum calculation.
mean	Arithmetic mean of pixel values within aperture.
median	Median of pixel values.
mode	$3 \times$ median $- 2 \times$ mean.
std	Standard deviation of pixel values.
var	Variance.
mad_std	Median absolute deviation, scaled to σ equivalent.
biweight_location	Robust biweight location M-estimator.
biweight_scale	Robust biweight scale.
min, max	Minimum and maximum pixel values in aperture.
fwhm	FWHM estimate from second-moment fit.
semimajor_sigma	Semi-major axis of 1σ ellipse (second moments).
semiminor_sigma	Semi-minor axis.
orientation	Position angle of major axis.
eccentricity	Eccentricity of the moment ellipse.

11.2.3 Example

```python
from photutils.aperture import ApertureStats, CircularAperture
from photutils.datasets import make_4gaussians_image
from astropy.stats import sigma_clipped_stats

data = make_4gaussians_image()
_, median, _ = sigma_clipped_stats(data, sigma=3.0)
data -= median

aper = CircularAperture((150.0, 25.0), r=8.0)
stats = ApertureStats(data, aper)

print(f"Centroid:  ({stats.xcentroid:.3f}, {stats.ycentroid:.3f})")
print(f"Flux sum:  {stats.sum:.2f}")
print(f"Mean:      {stats.mean:.4f}")
print(f"FWHM:      {stats.fwhm:.3f} pix")

```

```
# Export to table for multiple sources
aper_multi = CircularAperture([(150,25),(220,75),(90,40)], r=8.0)
s_multi = ApertureStats(data, aper_multi)
print(s_multi.to_table(columns=['id','xcentroid','ycentroid','sum','fwhm']))
```

11.2.4 Centroid Re-centring

ApertureStats does NOT re-centre. The centroid is measured within the input aperture, but the aperture is not moved to the measured centroid. For the most accurate photometry, iterate: create a new aperture centred at the measured centroid, then re-run `ApertureStats`.

12 Step 6: Local Background Estimation with Annuli

This is the photutils implementation of the background-annulus technique described in Sections 4 and 5. The `ApertureStats` class applied to a `CircularAnnulus`, combined with sigma clipping, gives a per-source sigma-clipped median background.

```
import numpy as np
from astropy.stats import SigmaClip
from photutils.aperture import (CircularAperture, CircularAnnulus,
                                ApertureStats)
from photutils.datasets import make_100gaussians_image

data = make_100gaussians_image()   # background ~ 5 ADU

positions = [(145.1, 168.3), (84.5, 224.1), (48.3, 200.3)]

src_ap  = CircularAperture(positions, r=5.0)
bkg_ann = CircularAnnulus(positions, r_in=10.0, r_out=15.0)

# Step 1: sigma-clipped background per pixel in the annulus
sigclip   = SigmaClip(sigma=3.0, maxiters=10)
bkg_stats = ApertureStats(data, bkg_ann, sigma_clip=sigclip)
bkg_per_pixel = bkg_stats.median          # shape (N_sources,)

# Step 2: source statistics (no sigma clipping inside source aperture)
src_stats = ApertureStats(data, src_ap, sigma_clip=None)

# Step 3: subtract background contribution
total_bkg   = bkg_per_pixel * src_stats.sum_aper_area.value
flux_bkgsub = src_stats.sum - total_bkg

for i, f in enumerate(flux_bkgsub):
```

```
    print(f"Source {i+1}: flux = {f:.2f}, bkg/pix = {bkg_per_pixel[i]:.3f}")
```

12.1 The local_bkg Keyword

Pre-computed background values can be injected directly:

```
# If you have a 2D background model from Background2D:
from photutils.background import Background2D, MedianBackground
bkg2d = Background2D(data, (50, 50), bkg_estimator=MedianBackground())

# Evaluate background at source positions (simplified: use global median)
local_bkg_vals = np.full(len(positions), bkg2d.background_median)

aperstats = ApertureStats(data, src_ap, local_bkg=local_bkg_vals)
print("Bkg-subtracted fluxes:", aperstats.sum)
```

This approach avoids creating a full background-subtracted array, which is important for large memory-mapped mosaics.

13 Step 7: Error Propagation in Detail

The uncertainty in the aperture sum is:

$$\Delta F = \sqrt{\sum_{i \in A} (\sigma_{\mathrm{tot},i} \cdot w_i)^2} \tag{29}$$

where $\sigma_{\mathrm{tot},i}$ is the per-pixel uncertainty from the `error` array, $w_i \in [0, 1]$ are the aperture weights, and A is the set of unmasked pixels. Masked and non-finite pixels are excluded from both the sum and the error.

The total per-pixel error array is constructed using `calc_total_error()`:

```
from photutils.utils import calc_total_error

# data_bkgsub: background-subtracted image
# bkg.background_rms: 2D background RMS from Background2D
# effective_gain: electrons/ADU (or exposure time if data is in e-/s)
error = calc_total_error(data_bkgsub,
                         bkg.background_rms,
                         effective_gain=gain)
```

14 Step 8: Masks and Bad Pixels

```
import numpy as np

mask = np.zeros(data.shape, dtype=bool)
mask[data > 60000] = True        # cosmic rays / saturation
mask[:, 512]       = True        # known bad column

# Pass to photometry functions
phot   = aperture_photometry(data_bkgsub, aperture, mask=mask)
stats  = ApertureStats(data_bkgsub, aperture, mask=mask)
```

Non-finite values (NaN, $\pm\infty$) are **automatically masked** in all photutils functions. When an NDData container is passed, its embedded mask, uncertainty, and wcs are used automatically and the corresponding keyword arguments are ignored.

15 Step 9: WCS and Sky Coordinates

```
from astropy.wcs import WCS
from astropy.coordinates import SkyCoord
from astropy import units as u
from photutils.aperture import SkyCircularAperture,
    aperture_photometry

# Load WCS from FITS header
with fits.open('image.fits') as hdul:
    wcs = WCS(hdul[0].header)

# Define positions in sky coordinates
pos_sky = SkyCoord(ra=[83.82, 83.91]*u.deg, dec=[-5.39,
    -5.44]*u.deg)
sky_ap  = SkyCircularAperture(pos_sky, r=3.0*u.arcsec)

# WCS is required for sky apertures
phot = aperture_photometry(data_bkgsub, sky_ap, wcs=wcs)
# Output table includes 'sky_center' column
```

16 Step 10: Converting to Calibrated Magnitudes

Following the transformation framework of Section 5.8, once zero-point ZP has been determined from standard star observations:

```
import numpy as np

```

```
ZP = 25.0    # photometric zero-point in your chosen system

flux     = phot_table['aperture_sum'].value
flux_err = phot_table['aperture_sum_err'].value

mag     = -2.5 * np.log10(flux) + ZP
mag_err = (2.5 / np.log(10)) * (flux_err / flux)  # error propagation

for i in range(len(flux)):
    print(f"Source {i+1}: m = {mag[i]:.4f} +/- {mag_err[i]:.4f}")
```

Aperture correction in practice: If the measurement aperture is smaller than $\sim 4\times$FWHM, the missing flux in the PSF wings must be added back as a magnitude correction Δ (Equation 17), measured from a bright isolated star on the same frame. This is critical for accurate absolute photometry.

17 Full End-to-End Pipeline

```
"""
Complete aperture photometry pipeline.
Input: a calibrated (bias/dark/flat-corrected) FITS image.
"""
import numpy as np
import matplotlib.pyplot as plt
from astropy.io import fits
from astropy.wcs import WCS
from astropy.stats import SigmaClip, sigma_clipped_stats
from astropy.table import QTable

from photutils.background import Background2D, MedianBackground
from photutils.aperture import (CircularAperture, CircularAnnulus,
                                ApertureStats, aperture_photometry)
from photutils.utils import calc_total_error

# ----------------------------------------------------------------
# 1. Load calibrated image
# ----------------------------------------------------------------
with fits.open('science_image.fits') as hdul:
```

```
    data   = hdul[0].data.astype(float).copy()
    header = hdul[0].header.copy()
wcs  = WCS(header)
gain = float(header.get('GAIN', 1.0))

# ----------------------------------------------------------------
# 2. 2D background estimation and subtraction
# ----------------------------------------------------------------
sigma_clip = SigmaClip(sigma=3.0)
bkg = Background2D(data, box_size=(64, 64), filter_size=(3, 3),
                   sigma_clip=sigma_clip, bkg_estimator=MedianBackground())
data_bkgsub = data - bkg.background

# ----------------------------------------------------------------
# 3. Build total error array
# ----------------------------------------------------------------
error = calc_total_error(data_bkgsub, bkg.background_rms,
                         effective_gain=gain)

# ----------------------------------------------------------------
# 4. Define source positions and apertures
#    (positions may come from a catalog, source finder, etc.)
# ----------------------------------------------------------------
positions   = [(145.1, 168.3), (84.5, 224.1), (48.3, 200.3)]
src_ap      = CircularAperture(positions, r=5.0)
bkg_ann     = CircularAnnulus(positions,  r_in=10.0, r_out=15.0)

# ----------------------------------------------------------------
# 5. Plot apertures on the image
# ----------------------------------------------------------------
from astropy.visualization import ZScaleInterval
z = ZScaleInterval()
```

```
vmin, vmax = z.get_limits(data_bkgsub)
fig, ax = plt.subplots(figsize=(10, 8))
ax.imshow(data_bkgsub, origin='lower', vmin=vmin, vmax=vmax, cmap='gray')
src_ap.plot(ax, color='cyan', lw=1.5, label='Source')
bkg_ann.plot(ax, color='red',  lw=1.5, label='Background')
ax.legend(); plt.tight_layout()
plt.savefig('apertures_check.png', dpi=150)

# ----------------------------------------------------------------
# 6. Local background (sigma-clipped median in annulus)
# ----------------------------------------------------------------
sigclip   = SigmaClip(sigma=3.0, maxiters=10)
bkg_stats = ApertureStats(data_bkgsub, bkg_ann, sigma_clip=sigclip)
bkg_per_pixel = bkg_stats.median

# ----------------------------------------------------------------
# 7. Source photometry
# ----------------------------------------------------------------
src_stats = ApertureStats(data_bkgsub, src_ap,
                          sigma_clip=None, error=error)
total_bkg   = bkg_per_pixel * src_stats.sum_aper_area.value
flux_clean  = src_stats.sum - total_bkg

# ----------------------------------------------------------------
# 8. Aperture correction (measured from a bright star, simplified)
# ----------------------------------------------------------------
bright_pos    = [(150.0, 25.0)]
bright_small  = ApertureStats(data_bkgsub, CircularAperture(bright_pos, r=5.0))
bright_large  = ApertureStats(data_bkgsub, CircularAperture(bright_pos, r=20.0))
delta_apcorr  = -2.5 * np.log10(bright_large.sum / bright_small.sum)
print(f"Aperture correction: {delta_apcorr[0]:.4f} mag")

```

```
# ------------------------------------------------------------------
# 9. Instrumental magnitude -> calibrated magnitude
# ------------------------------------------------------------------
ZP      = 25.0
mag     = -2.5 * np.log10(flux_clean) + ZP + delta_apcorr[0]
mag_err = (2.5 / np.log(10)) * (src_stats.sum_err / flux_clean)

# ------------------------------------------------------------------
# 10. Assemble output table
# ------------------------------------------------------------------
results = QTable({
    'id':           np.arange(1, len(positions) + 1),
    'x':            [p[0] for p in positions],
    'y':            [p[1] for p in positions],
    'flux':         flux_clean,
    'flux_err':     src_stats.sum_err,
    'bkg_per_pix': bkg_per_pixel,
    'magnitude':    mag,
    'mag_err':      mag_err,
    'xcentroid':    src_stats.xcentroid,
    'ycentroid':    src_stats.ycentroid,
    'fwhm':         src_stats.fwhm,
})
results.write('photometry_results.fits', overwrite=True)
print(results)
```

18 Common Pitfalls

The following are the most frequent mistakes in aperture photometry, drawn from both Romanishin's chapter on "Common (and Un-Common) Photometry Goofups" and practical `photutils` experience:

1. **No background subtraction.** Both `aperture_photometry()` and `ApertureStats` assume background-subtracted data. Failure to subtract produces inflated fluxes equal to: true flux + $N_{\text{pix}} \times$ bkg/pixel.
2. **Using the mean instead of the median for the sky.** The mean is biased by contaminating sources and cosmic rays. Always use the sigma-clipped median in the background annulus.

3. **Not sigma-clipping the annulus.** Even one bright contaminant in the annulus causes the sky to be overestimated, leading to flux underestimation after background subtraction.
4. **Aperture too small without aperture correction.** If the aperture is smaller than $\sim 4\times$FWHM, significant source flux lies in the PSF wings outside the aperture. This fraction varies with seeing conditions and can produce inconsistent results from frame to frame.
5. **Aperture too large.** Unnecessarily large apertures include more sky noise ($\sigma \propto \sqrt{N_{\rm pix}}$) and risk contamination from nearby sources.
6. **Annulus overlapping source.** The inner radius of the background annulus must be larger than the source aperture radius by at least 2–3 pixels to avoid the PSF wings biasing the sky estimate.
7. **Pixel coordinate confusion.** `photutils` uses 0-indexed (x, y) coordinates (Python convention); FITS and DS9 use 1-indexed conventions. Subtract 1 from FITS coordinates before passing to `photutils`.
8. **Modifying memory-mapped arrays without copying.** Subtracting the background in-place on a memory-mapped FITS array either modifies the file on disk or raises an error. Always `.copy()` before modifying.
9. **Sky apertures without a WCS.** `SkyCircularAperture` and other sky apertures will raise an error if no `wcs=` is provided to the photometry function.
10. **Not checking for non-photometric conditions.** "Many people don't believe the assertion above, that your eye cannot detect thin clouds at a dark place. I have spent far too many nights on Kitt Peak observing away all night, thinking it was clear, only to find thin clouds revealed as the sky started to lighten." Always monitor the sky transparency quantitatively (e.g., by tracking the measured flux of bright standard stars over time).
11. **Scattered light in flat fields.** Flat fields taken with scattered light will not correctly calibrate star images, introducing systematic errors in photometry that look subtle but are real.

19 Quick Reference

19.1 Tool Selection

Task	Tool
Simple flux sum	`aperture_photometry()`
Flux + centroids + FWHM + morphology	`ApertureStats`
Flat uniform sky	`sigma_clipped_stats` → scalar subtraction
Spatially varying sky	`Background2D`
Local per-source sky	`CircularAnnulus` + `ApertureStats(sigma_clip=)`
Error propagation	`calc_total_error()` + pass `error=`
Bad pixel masking	Pass `mask=` boolean array
Sky-coordinate apertures	`Sky*` aperture classes + `wcs=`
Large mosaics	`local_bkg=` keyword in `ApertureStats`
Precise boundary pixels	`method='exact'` (default)
Fast approximate measurement	`method='center'`

19.2 Aperture Size Guidelines (Summary from Romanishin)

Aperture purpose	Recommended radius
Optimal S/N (faint sources)	$\sim 1.0\times$FWHM
Best practical aperture correction	$\sim 0.7\times$FWHM (diameter $1.4\times$FWHM)
Large aperture (capture all flux)	$\sim 4\times$FWHM
Background annulus inner edge	$> 1.5 \times r_{\rm ap}$ (gap from source)
Background annulus outer edge	$\sim 1.3 \times r_{\rm in}$ to $1.5 \times r_{\rm in}$

20 Further Reading

- **Romanishin, W.** (2002). *An Introduction to Astronomical Photometry Using CCDs*. University of Oklahoma. Available at http://observatory.ou.edu. The primary conceptual reference for this document.
- **Howell, S. B.** (1989). "Two-dimensional aperture photometry: Signal-to-noise ratio of point-source observations and optimal data-extraction techniques." *PASP*, 101, 616. Foundational paper on SNR in aperture photometry and optimal aperture size.
- **Harris, W. E.** (1990). "Aperture photometry in crowded fields." *PASP*, 102, 949.
- **photutils documentation:** https://photutils.readthedocs.io/en/stable/

- **Aperture photometry user guide:** https://photutils.readthedocs.io/en/stable/user_guide/aperture.html
- **Background estimation user guide:** https://photutils.readthedocs.io/en/stable/user_guide/background.html
- **ApertureStats API reference:** https://photutils.readthedocs.io/en/stable/api/photutils.aperture.ApertureStats.html